

Как приложение и сервер узнают друг друга

Документ описывает, как приложение NOX и клиентский сервер проверяют друг друга и защищают соединение: **что умеют технологии** на пяти платформах, где хранятся ключи и как устроены проверка и шифрование.

ПЛАТФОРМЫ

Android

iOS

macOS

Windows

Linux

■ приложение, устройство ■ клиентский сервер ■ не работает

Возможности

Решение коротко

Dart и Rust

Хранение ключей

Взаимный TLS

Библиотеки приложения

Библиотеки сервера

Термины

Технические возможности

TLS — защитная оболочка соединения: договаривается о ключе шифрования, шифрует данные и даёт сторонам доказать, кто они. **Ed25519** — алгоритм подписи: им сторона доказывает, что ключ — её. TLS реализуют библиотеки; ниже — три из них и что каждая умеет.

Что сравнивается

БИБЛИОТЕКА	ЧТО ЭТО	ВЕРСИЯ	ГДЕ
dart:io	TLS из стандартной библиотеки Dart, внутри — BoringSSL от Google. Через неё работают HttpClient, Dio и WebSocket во Flutter-приложении	входит в Dart SDK; актуальная — 3.13.5, проверено на 3.12	api.dart.dev
rustls	библиотека TLS на Rust с открытым кодом, подключается к приложению в Rust-модуле	0.23.45 — сентябрь 2026	crates.io · GitHub
crypto/tls	пакет TLS из стандартной библиотеки Go: ставится вместе с Go, отдельно не подключается	Go 1.27.1 — сентябрь 2026	pkg.go.dev

Что они умеют

ЧТО НУЖНО	DART:IO (DART)	RUSTLS (RUST)	CRYPTO/TLS (GO)
TLS 1.3	да	да	да
Сертификат с ключом Ed25519	нет в BoringSSL выключен, Dart не даёт включить	да	да с Go 1.13
Сертификат с ключом ECDSA P-256	да	да	да
Взаимная проверка: сертификат с обеих сторон	да с ключами RSA и ECDSA	да в том числе Ed25519	да в том числе Ed25519
Пять платформ	одинаково	одинаково	—

■ Откуда TLS во Flutter

Из встроенной библиотеки Dart `dart:io`, внутри которой — BoringSSL от Google, та же, что в Chrome. Своего TLS у Dio и WebSocket нет: они работают через неё, одинаково на всех пяти платформах.

■ Почему не работает Ed25519

BoringSSL умеет Ed25519, но по умолчанию выключает его, а Dart включить не даёт. Сервер с сертификатом Ed25519 не находит общего типа подписи и рвёт соединение раньше, чем приложение успеет что-то проверить.

Решение коротко

Дальше документ описывает одно решение — **взаимный TLS 1.3 на ключах Ed25519**. Это одна стандартная процедура, которая и шифрует соединение, и проверяет обе стороны. Реализуют её две стороны — приложение и сервер.

■ Приложение

TLS делает Rust-модуль приложения на библиотеке rustls: встроенный в Dart TLS не умеет Ed25519. Команды, WebSocket и файлы живут в Dart и работают поверх модуля. Ключ устройства Ed25519 хранится в системном сейфе.

■ Соединение

TLS 1.3 по любому пути — дома, по внешнему адресу, через Tor. Обе стороны показывают свои ключи Ed25519 и подписывают рукопожатие, после чего всё шифруется общим ключом.

■ Сервер

TLS делает стандартная библиотека Go с ключом сервера Ed25519. Ключ устройства сервер сверяет со списком спаренных устройств.

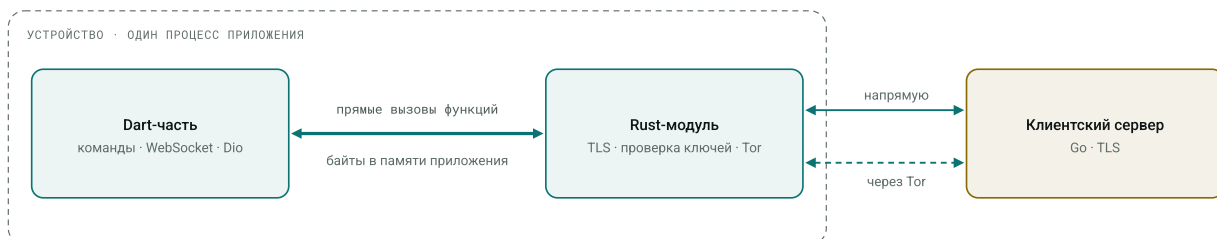
ОТКУДА СТОРОНЫ ЗНАЮТ КЛЮЧИ ДРУГ ДРУГА

Из спаривания: ключ сервера приходит в приложение в QR-коде, а сервер записывает ключ устройства по одноразовому токenu.

Дальше по порядку: как Dart-часть приложения связана с Rust-модулем, где хранятся ключи, как проходит проверка в TLS 1.3 и какие библиотеки используются.

TLS в Rust: как с ним связан Dart

Rust-модуль — часть приложения. Он собирается вместе с приложением для каждой из пяти платформ и работает в том же процессе.



Dart-часть и Rust-модуль работают в одном процессе приложения и передают байты друг другу в памяти.

Прямые вызовы функций. Dart вызывает функции Rust-модуля: «открой соединение», «отправь байты», «закрой». Входящие байты модуль возвращает в Dart обратным вызовом. Байты передаются в памяти процесса и не выходят за пределы приложения.

Свой сокет для WebSocket и Dio. В Dart соединение — объект `Socket`. Приложение подставляет свой `Socket`, который отправляет и получает байты через вызовы Rust-модуля. `HttpClient` принимает такой сокет, поэтому WebSocket и Dio работают поверх него как обычно. Проверено по исходникам Dart SDK 3.12: `Socket` открыт для реализации, а `HttpClient` обращается к нему только через общие методы.

Выше — Dart. WebSocket, команды, Dio для файлов, очередь отправки живут в Dart и получают от Rust-модуля готовое проверенное и зашифрованное соединение.

Путь через Tor — тем же способом. Tor-клиент находится в том же Rust-модуле: Dart так же передаёт ему байты, а модуль отправляет их через Tor.

реализуемо

Свой `Socket` поверх вызовов Rust-модуля — `HttpClient`, WebSocket и Dio его принимают; проверено по исходникам Dart SDK 3.12.

реализуемо

Путь через Tor тем же способом — Tor-клиент в том же Rust-модуле.

Где хранятся ключи на устройствах

Закрытый ключ устройства можно держать двумя способами, и от способа зависит, какие алгоритмы доступны.

■ Системный сейф

любой алгоритм

Защищённое хранилище системы для любых секретов: паролей, токенов, ключей любого типа, в том числе Ed25519. Данные лежат зашифрованными, чужие приложения их не читают. Для подписи приложение на миг берёт ключ в свою память.

■ Ключ внутри чипа

только алгоритмы чипа

Ключ создаётся в аппаратном чипе безопасности и никогда его не покидает: приложение только просит чип подписать. Скопировать такой ключ нельзя даже со взломанного устройства. Но чип умеет лишь те алгоритмы, что в него заложены.

ПЛАТФОРМА	СИСТЕМНЫЙ СЕЙФ	ЧИП	ED25519 В ЧИПЕ	P-256 В ЧИПЕ
iOS	Keychain	Secure Enclave	нет	да
macOS	Keychain	Secure Enclave — Mac на Apple Silicon или с чипом T2	нет	да
Android	хранилище, зашифрованное ключом из Android Keystore	защищённая часть процессора; у части устройств — StrongBox	с Android 13, не везде в StrongBox — обычно нет	да

ПЛАТФОРМА	СИСТЕМНЫЙ СЕЙФ	ЧИП	ED25519 В ЧИПЕ	P-256 В ЧИПЕ
Windows	Credential Manager	TPM 2.0, если есть	нет	да
Linux	Secret Service: GNOME Keyring, KWallet	TPM 2.0, если есть; приложению добраться сложно	нет	да

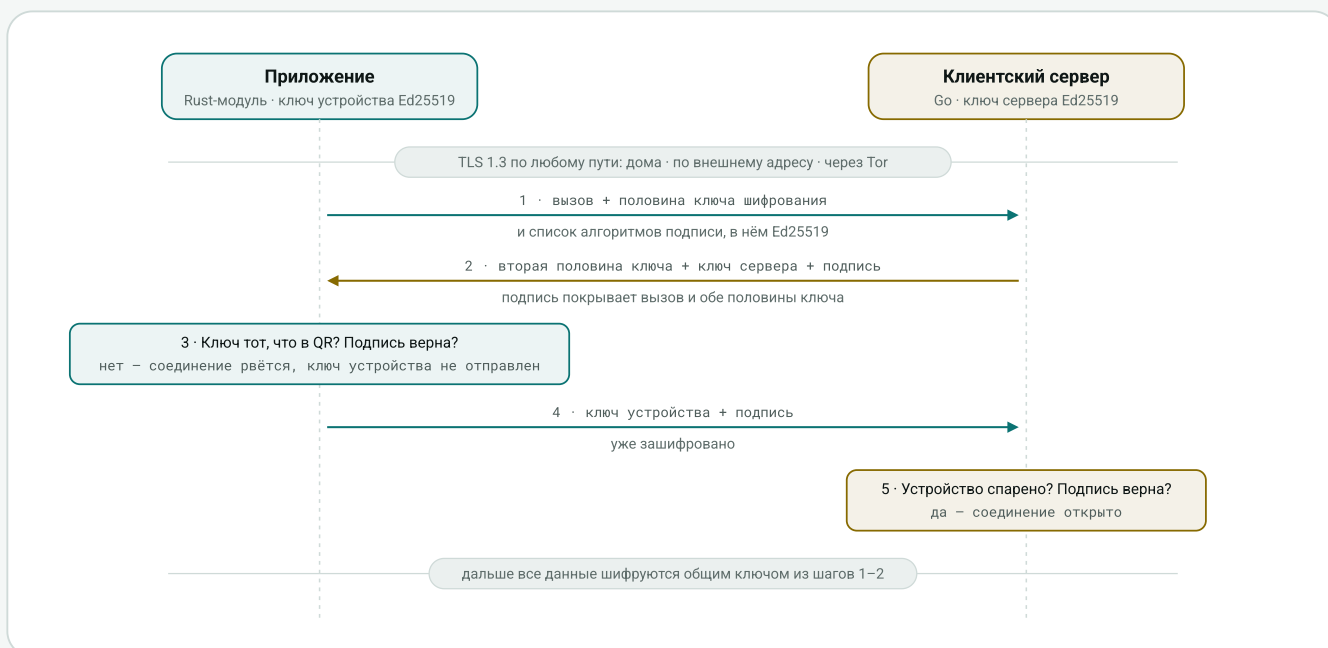
КЛЮЧ УСТРОЙСТВА

Ключ устройства Ed25519 хранится в системном сейфе — одинаково на пяти платформах. Хранение в чипе можно добавить позже, но только с ключом P-256: Ed25519 чипы iPhone, Mac и Windows не поддерживают, на Android — не все.

На Linux сейф работает, только если в системе запущена служба ключей — GNOME Keyring или KWallet.

Взаимный TLS на Ed25519

У сервера и у каждого устройства — по одному ключу Ed25519. TLS делают rustls в приложении и стандартная библиотека Go на сервере. Обе умеют Ed25519, поэтому взаимную проверку выполняет сам TLS 1.3.



Обе стороны показывают ключи и подписывают свежие данные друг друга. Это выполняет стандартный TLS 1.3.

Шаги проверки

ШАГ	ЧТО ПРОИСХОДИТ	ЗАЧЕМ
1. Вызов	Приложение шлёт случайный вызов и половину ключа шифрования	вызов новый на каждое соединение, поэтому записанную подпись нельзя повторить
2. Ответ сервера	Сервер шлёт свой ключ Ed25519 и подпись, которая покрывает вызов и обмен ключом шифрования	сервер доказывает, что ключ его, именно в этом соединении
3. Проверка сервера	Приложение сверяет ключ с ключом из QR и проверяет подпись	чужая машина отсеивается раньше, чем приложение что-то сообщит о себе
4. Ответ приложения	Приложение шлёт свой ключ Ed25519 и подпись — уже зашифрованно	устройство доказывает, что ключ его; посторонний этот ключ не видит
5. Проверка устройства	Сервер ищет ключ среди спаренных устройств и проверяет подпись	в соединение попадают только свои устройства

ШИФРОВАНИЕ И ПРОВЕРКА — ОДИН ПРОЦЕСС

Ключ шифрования стороны согласуют в тех же сообщениях, в которых проверяют друг друга, и подписи покрывают этот обмен. Поэтому подменить ключ шифрования, не сломав подписи, нельзя, а зашифрованный канал заведомо принадлежит тому, кого проверили. Отдельного шага «войти» поверх TLS нет.

Что делает TLS, а что остаётся нам

ДЕЛАЕТ TLS	ОСТАЁТСЯ НАМ
Проверка сторон: вызовы и подписи Ed25519 с обеих сторон	Решить, свой ли это ключ: в приложении — тот ли, что в QR; на сервере — есть ли он среди спаренных устройств
Договориться о ключе шифрования	Спаривание: ключ сервера попадает в приложение через QR, ключ устройства — на сервер по токenu
Зашифровать все данные	Создать ключи, хранить их в системном сейфе, обернуть ключ в сертификат
Защитить от повтора, подмены и перехвата	Отзывать устройства и показывать человеку, почему отказано
	Строгая настройка: только TLS 1.3, только Ed25519, сертификат устройства обязателен

Так же устроен браузер: криптографию делает TLS, а кому верить, решает браузер по удостоверяющим центрам. У нас вместо них — QR-код и список

спаренных устройств на сервере.

Спаривание. QR несёт адрес сервера, его ключ — 32 байта — и одноразовый токен. Новое устройство проходит то же рукопожатие, но в списке спаренных его ещё нет, поэтому сервер пускает такое соединение только для одного — предъявить токен. После этого ключ устройства записан: владение им устройство уже доказало подписью.

Путь не важен. То же рукопожатие идёт по любому пути — дома, по внешнему адресу или через Tor.

Ключ устройства лежит в системном сейфе, а Rust-модуль берёт его для подписи рукопожатия.

Связь Dart и Rust — прямыми вызовами функций внутри одного процесса приложения.

реализуемо

Ed25519 в TLS с обеих сторон: Go — с версии 1.13, rustls — да; проверено по документации обеих библиотек.

реализуемо

Rust-модуль на всех пяти платформах — штатная сборка Flutter (native assets); инструмент сборки Rust поддерживает Android, iOS, macOS, Windows и Linux.

с оговорками

Ключ устройства в чипе — Rust-модулю пришлось бы обращаться к чипу на каждой платформе, и только с P-256.

Библиотеки приложения

Flutter-приложение и его Rust-модуль. Версии — актуальные на 7 октября 2026 года.

ЗАДАЧА	БИБЛИОТЕКА	ВЕРСИЯ	ГДЕ	ПОСТАВЩИК · ПРОВЕРКА
TLS	rustls	0.23.45	crates.io · GitHub	проект rustls; аудит Cure53, 2020 — только низкие и информационные замечания
Криптография под TLS	ring	0.17.14	crates.io · GitHub	входил в тот же аудит Cure53, 2020
Сертификат устройства	rcgen	0.14.10	crates.io · GitHub	проект rustls
Сборка Rust-модуля во Flutter	native_toolchain_rust	1.0.7	pub.dev	Android, iOS, macOS, Windows, Linux

ЗАДАЧА	БИБЛИОТЕКА	ВЕРСИЯ	ГДЕ	ПОСТАВЩИК · ПРОВЕРКА
Связь Dart и Rust	dart:ffi и свой Socket в Dart	входит в Dart SDK	dart.dev	часть Dart
Хранение ключа устройства	flutter_secure_storage	11.2.0	pub.dev	системные сейфы: Keychain, Android Keystore, Credential Manager, Secret Service

ЕСЛИ ПОНАДОБЯТСЯ ПОДПИСИ ВНЕ TLS

Например, для спаривания по короткому коду вместо QR — ed25519-dalek 3.0.0 на Rust (crates.io). Его пишут авторы ristretto255, которых указывает ristretto.group; аудит Quarkslab, 2019 — критических уязвимостей нет.

Библиотеки сервера

Клиентский сервер на Go. Всё нужное — в стандартной библиотеке Go: ставится вместе с Go, сторонней криптографии нет. Версии — актуальные на 7 октября 2026 года.

ЗАДАЧА	БИБЛИОТЕКА	ВЕРСИЯ	ГДЕ	ПОСТАВЩИК · ПРОВЕРКА
TLS	crypto/tls	Go 1.27.1	pkg.go.dev	команда Go; Ed25519 в TLS — с Go 1.13
Сертификат сервера и разбор сертификатов устройств	crypto/x509	Go 1.27.1	pkg.go.dev	команда Go
Ключи и подписи Ed25519	crypto/ed25519	Go 1.27.1	pkg.go.dev	команда Go; аудит Trail of Bits, 2025; сертификат FIPS 140-3 № 5247

Термины

TLS защитная оболочка соединения: ключ шифрования, шифрование, проверка сторон. Стоит за https и wss.

Сертификат открытый ключ стороны в стандартной обёртке, которую показывают в TLS.

Ed25519 алгоритм подписи. Ключ — 32 байта, подпись — 64 байта.

ECDSA P-256 другой алгоритм подписи, старше и распространённое; единственный, который умеют все чипы.

Вызов случайные байты, свежие на каждое соединение. Подпись над ними доказывает, что ключ у стороны есть сейчас.

Рукопожатие первые сообщения соединения: стороны проверяют друг друга и договариваются о ключе шифрования.

Общий ключ ключ шифрования одного соединения. Стороны договариваются о нём в рукопожатии; после соединения он уничтожается.

Спаривание однократное знакомство устройства с сервером по QR-коду: приложение получает ключ сервера, сервер записывает ключ устройства.

Прямые вызовы (FFI) способ, которым Dart-часть вызывает функции Rust-модуля внутри одного процесса приложения.

Сокет объект соединения в Dart: WebSocket и Dio пишут и читают байты через него.

Системный сейф защищённое хранилище секретов системы: Keychain, Android Keystore, Credential Manager, Secret Service.

Чип безопасности Secure Enclave, StrongBox, TPM: держит ключ внутри и сам подписывает.